

התפתחות מנגנוני תיאום כלים מרובים בסוכני LLM

אבי איילי — ת.ז. 300228160
קורס: אורכסטריציה של סוכני AI
מרצה: ד"ר יורם סגל
2026

מסמך זה נוצר בסיוע בינה מלאכותית

תוכן העניינים

2	1 מבוא לחילוץ אותות מרעש
2	1.1 בעיית חילוץ הגלים הסינסואידליים
2	1.2 גישות מסורתיות וליקויים שלהן
3	1.3 למה למידה עמוקה?
3	1.4 יסודות NNR ו-MTSL לעיבוד סדרתי
3	1.4.1 רשתות נירונים חוזרות (sNNR)
4	1.4.2 יחידות זיכרון ארוך טווח (MTSL)
4	1.4.3 מנגנוני קשב בסדרות
5	1.4.4 sMTSL דו-כיווניים (MTSLiB)
6	2 ניהול מצב דו-כיווניות טקסט
6	2.1 מנגנוני ניהול מצב בעבודה עם מודלים
6	2.2 דו-כיווניות בעיבוד טקסט
8	3 פרוטוקול בקרת המודל
8	3.1 אינטרפייס של Model Context Protocol
8	3.2 טכניקות שדרוג קשר
8	3.3 ביטחון וערכיות בשדרוג קשר
01	4 איומים וחולשות
01	4.1 סוגי איומים על מודלים של שפה
01	4.2 איומים על פרוטוקול בקרת המודל
01	4.3 אתגרים של קנה מידה
11	5 הפחתת סיכונים וכיווניות עתידית
11	5.1 אסטרטגיות הפחתת סיכונים
11	5.2 תקנון וחקיקה
21	5.3 כיווניות עתידית ומחקר פתוח
31	נספח א': מילון מונחים

פרק 1

מבוא לחילוץ אותות מרעש

1.1 בעיית חילוץ הגלים הסינוסואידליים

בעולם התקשורת וממיד אותות, מתמודדים בעיה יסודית: חילוץ גלים סינוסואידליים מתוך תערובת רועשת של אותות. במצבים מעשיים רבים, האות הרצוי (evawenis) משודרת דרך ערוץ תקשורת או חיישן פיזי, אך מתקבלת מזוהמת בתוספת רעש אקראי. המטרה היא לשחזר את הגלים הסינוסואידליים המקוריים, תוך דחיקת התרומה הרועשת.

באופן פורמלי, ניתן לתאר את הבעיה כך: בהינתן סדרת תצפיות $\mathbf{y} = [y_1, y_2, \dots, y_N]$, כאשר

$$(1.1) \quad y_t = \sum_{k=1}^K A_k \sin(2\pi f_k t + \phi_k) + n_t$$

כאן A_k היא אמפליטודה, f_k היא התדירות, ϕ_k היא הפאזה של הגל ה- k , ו- n_t הוא רעש תוספי (בדרך כלל גאוסיאני לבן). המטרה היא לשחזר את $\{A_k, f_k, \phi_k\}$ בצורה מדויקת, אפילו כאשר יחס האות-לרעש (RNS) נמוך מאוד.

יישומים מעשיים לבעיה זו מופיעים בתחומים מגוונים: בטלקומוניקציה, במערכות מקום (SPG), בעיבוד אותות אקוסטיים, ובחיזוי סדרות זמן פיננסיות. בכל היישומים הללו, ההבדל בין שחזור מדויק לשחזור גרוע עלול להשפיע משמעותית על ביצועי המערכת כולה.

1.2 גישות מסורתיות וליקויים שלהן

ההתמודדות עם בעיה זו אינה חדשה. לעדי מאות שנים, עוסקים בעיבוד אותות בטכניקות קלסיות המבוססות על ניתוח פורייה ותיאוריה של עיבוד אותות ליניארי. אולם גישות אלו הוכיחו מגבלות משמעותיות בהקשר של אותות לא-סטציונריים ורעש בעוצמה גבוהה.

Fast Fourier Transform (TFF), על אף שהינה כלי חזק, מניחה שהאות הוא סטציונרי (כלומר תכונותיו אינן משתנות בזמן) וכי רעשים מופץ אחידות על כל הטווח התדרי. בסביבות תדר נמוכה או כאשר האות מכיל מרכיבים בתדרים קרובים זה לזה, ה-TFF סובלת מחוסר דיוק. בנוסף, שיטות עיבוד ליניארי (כגון תיאוריה קלסית של סינון Wiener filtering) דורשות ידע קודם על מאפייני הרעש, וביצועיהן נחותים כאשר התדרים בעלי שונות או בעלי קשרים לא-ליניאריים.

גישות קלסיות נוספות, כגון אומדן דיוק מקסימלי (Maximum Likelihood Estimation) או שיטות least-squares, דורשות פתרון בעיות אופטימיזציה לא-קמורות מורכבות וממצאים מקומיים, במיוחד כאשר מספר הגלים K אינו ידוע מראש. בנוסף, זמן החישוב שלהן גדל בחזקה עם אורך הסדרה וכמות הנתונים.

1.3 למה למידה עמוקה?

בעשור האחרון, הראו רשתות נוירונים עמוקות יכולת ייחודית בלימוד דפוסים מורכבים משום שרשתות אלו יכולות ללמוד ייצוגים (snoitatheserper) לא-ליניאריים של הנתונים בצורה מדורגת. בהקשר של חילוף אותות מרעש, משפחת הרשתות (sNNR) Recurrent Neural Networks, בעיקר Long Short-Term Memory (MTSL), הוכיחו יעילות משמעותית. למידה עמוקה מציעה מספר יתרונות מהותיים על פני גישות קלאסיות:

1. **למידה של תלויות ארוכות טווח:** sNNR ו-sMTSL יכולות ללמוד דפוסים זמניים המשתרעים על אלפי צעדי זמן, ללא צורך בידע קודם על מבנה הרעש או מאפייני האות.
 2. **שיוך דפוסים לא-ליניאריים:** רשתות נוירונים עמוקות מצטיידות יכולת לגלות קשרים לא-ליניאריים בין הנתונים, אשר אינם נתפסים על ידי סינוני ליניאריים קלאסיים.
 3. **דיוק גבוה בתנאים קיצוניים:** מחקרים הראו כי sMTSL משיגים ביצועי שחזור עדיפים בתנאי RNS נמוך מאוד (עד $Bd - 20$) בהשוואה לשיטות TFF או Wiener filtering.
 4. **הסתגלות ויכולת הכללה:** בניגוד לשיטות קלאסיות, רשתות עמוקות המאומנות על הרבה דוגמאות יכולות להכליל למצבים שלא ראו בעבר, בתנאי שלמערכות הבדיקה מאפיינים דומים.
- בפרקים הקרובים, נחקור את יסודות NNR ו-MTSL, נרחיב על יישומים ב-PyTorch, ונדון בארכיטקטורות מתקדמות להשגת ביצועים אופטימליים בחילוף אותות סינוסואידליים מתוך רעש כבד.

1.4 יסודות NNR ו-MTSL לעיבוד סדרתי

1.4.1 רשתות נוירונים חוזרות (sNNR)

רשתות נוירונים חוזרות (sNNR, skrowteN larueN tnerruceR) מהוות את הבסיס של עיבוד סדרות זמניות בלמידה עמוקה. בניגוד לרשתות עצביות מלאות חיבור (skrowten drawrofdeef) שמעבדות כל קלט באופן עצמאי, sNNR שומרות על hidden state שמתעדכן בכל צעד זמני. המצב הסמוי (etats neddih) משמש כ"זיכרון" של הרשת, המאפשר לה להשפיע על התפוקה הנוכחית על סמך קלטים קודמים. הארכיטקטורה הבסיסית של NNR מוגדרת כך:

$$(1.2) \quad h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

כאשר h_t הוא המצב הסמוי בזמן t , x_t הוא הקלט בזמן t , W_{hh} היא מטריצת המשקלים מהמצב הסמוי לעצמו, W_{xh} היא מטריצת המשקלים מהקלט למצב הסמוי, ו- b_h הוא הביאס. התפוקה y_t מחושבת כרגיל מהמצב הסמוי:

$$(1.3) \quad y_t = W_{hy}h_t + b_y$$

היתרון המרכזי של sNNR הוא יכולתן לעבד סדרות באורך משתנה ולתפוס תלויות זמניות. עם זאת, לארכיטקטורה זו יש חסרון קיטלי: בעיית vanishing gradient. כאשר הרשת מתאמנת באמצעות backpropagation through time (BPTT), הגרדיאנטים מתפשטים לאחור דרך שרשרת של מטריצות חזקה. אם הערכים העצמיים של W_{hh} קטנים מ-1, הגרדיאנטים דועכים באופן אקספוננציאלי, מה שהופך קשה מאוד ללמוד תלויות ארוכות טווח. במעשה, כאשר מנסים לחזות את ערכי האות הסינוסואידאלי בעתיד, חשוב מאוד להזכור את הדפוסים של גלים קודמים. sNNR בסיסי לא יכול לעשות זאת בצורה יעילה, מכיוון שהמידע מזמנים רחוקים מתחזק מהר מדי. לכן, הוצגה ארכיטקטורה משופרת: Long Short-Term Memory (LSTM).

1.4.2 יחידות זיכרון ארוך טווח (MTSL)

LSTM היא הרחבה של NNR שנועדה להתמודד עם בעיית הגרדיאנט הדועך. במקום מצב סמוי יחיד, ל-MTSL יש תא זיכרון (cell state) שנשמר באופן קבוע כל העת. תא זיכרון זה מתעדכן דרך שלוש gates (שערים) המסדירים את זרימת המידע:

$$(1.4) \quad f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

השער הראשון הוא forget gate, שמחליט איזה חלקים מתא הזיכרון הקודם להשליך. σ היא פונקציית sigmoid, המחזירה ערכים בטווח $[0, 1]$.

$$(1.5) \quad i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$(1.6) \quad \tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

השער השני הוא input gate, שמחליט איזה ערכים חדשים להכניס לתא הזיכרון. $\tilde{C}_t$ הם ערכי המועמדות החדשים שמכניסים לתא.

$$(1.7) \quad C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

כאן מעדכנים את תא הזיכרון: משקללים את הערכים הישנים בשער הזיכרון (f_t) וחיברים את הערכים החדשים ($i_t \odot \tilde{C}_t$).

$$(1.8) \quad o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$(1.9) \quad h_t = o_t \odot \tanh(C_t)$$

השער השלישי הוא output gate, שמחליט איזה חלקים של תא הזיכרון לנטר ליוצא הדופן. המצב הסמוי החדש h_t נחשב מתוך השער וההטרנספורמציה של תא הזיכרון. היתרון של MTSL הוא שתא הזיכרון מעודכן בתוספת (etadpu evitidda) כנגד הכפלה במטריות משקלים. כתוצאה מכך, הגרדיאנט הזורם דרך תא הזיכרון נשמר ללא דעיכה אקספוננציאלית. זה מאפשר ללמידה ארוכת טווח לעבוד בצורה הרבה יותר יעילה. בהקשר של חילוף גלים סינכרוניים מאותות רועשות, MTSL יכול ללמוד בקלות לזהות את הדפוסים התקופתיים של הגל. הרשת יכולה לשמור במצב הסמוי מידע אודות הטווח שעבר וההנחתה של הגל, ולהשתמש בו כדי לחזות את הערכים העתידיים.

1.4.3 מנגנוני קשב בסדרות

Attention mechanisms הם כלים חזקים שמאפשרים לרשת למדוד איזה חלקים של הקלט חשובים ביותר לתפוקה הנוכחית. במודל Transformer של inawsaV וחברים [noitnetta7102inawsav], תנגנון קשב זה משמש כבסיס עיקרי. מנגנון Scaled Dot-Product Attention מוגדר כ:

$$(1.10) \quad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

כאשר Q היא מטריצת השאלות (Query), K היא מטריצת המפתחות (Key), ו- V היא מטריצת הערכים (Value). כל השלוש מטריצות אלה נלמדות במהלך האימון. $\sqrt{d_k}$ הוא גורם נורמליזציה, כאשר d_k הוא מימדיות המפתחות. הרעיון הוא שהקשב מחשב עקביות בין כל זוג של עמדות בסדרה, ואחר כך משקללת את הערכים לפי עקביות אלה. בדרך זו, הרשת יכולה focus על הווידאו החשוב ולהתעלם מאחרים.

בהקשר של סדרות זמניות ועיבוד אותות, קשב עוזר לרשת ללמוד אילו צעדים בעבר החשובים ביותר לחיזוי הנקודה הבאה. עבור גלים סינוסואידאליים, לדוגמה, הרשת יכולה ללמוד שהערכים האחרונים חשובים יותר מאלה שהיו קדימה בעבר.

1.4.4 sMTSL דו-כיווניים (MTSLiB)

עד כה דברנו על sMTSL שמעבדים סדרות בכיוון אחד בלבד (מקדימה לאחור בזמן). Bidirectional LSTM (BiLSTM) מוסיפה MTSL שני שמעבד את הסדרה בכיוון ההפוך. שתי הרשתות מעבדות את אותו קלט באופן סימולטני, והתפוקות שלהן משולבות (בדרך כלל על ידי שרשור).

$$(1.11) \quad h_t^{\text{BiLSTM}} = [h_t^{\text{forward}}; h_t^{\text{backward}}]$$

כאן h_t^{forward} הוא המצב הסמוי של ה-MTSL שמעבד את הסדרה מקדימה, ו- h_t^{backward} הוא המצב הסמוי של ה-MTSL שמעבד את הסדרה לאחור. הסימן ; מייצג שרשור וקטורים. תופעה מעניינת ודברים בבילוגיה: כאשר אנו עומדים מול אות רועשה, אנו יכולים להביט קדימה ואחורה כדי להבין מה קורה בנקודה הנוכחית. זוהי בדיוק מה שעושה MTSLiB. זה מאפשר למודלים לעשות זאת, במיוחד כאשר מעבדים את הנתונים במצב אופליין (כלומר, כאשר כל הנתונים זמינים מראש).

עם זאת, MTSLiB אינו מתאים לחיזויים בזמן אמת (real-time forecasting), שכן הוא דורש גישה לנתונים עתידיים. לחיזויים בזמן אמת, אנו משתמשים ב-MTSL רגיל או unidirectional RNN. סיכום פרק זה: sNNR בסיסי סובל מבעיות גרדיאנט, sMTSL פותר זאת עם שערים וזיכרון תא. noitnetTA מגננונים מסייעים במודלים בחירה של קלטים חשובים. MTSLiB אוגר בקורה הקשר הדו-כיווני כדי ניתוח אופליין. בפרק הבא, נתור כיצד ליישם מגננונים אלה ב-hcroTyP.

פרק 2

יישום ב-hcroTyP: מבנה ותהליך הדרכה

2.1 בניית מודלים ב-hcroTyP

hcroTyP היא ספריית למידה עמוקה שהפכה למסגרת הבחירה לחוקרי עיבוד אותות בשל גמישותה ותמיכתה הניתב הדינמי. בעבודה עם סדרות זמן (seires emit), אנו מגדירים מודלים MTSL מקוננים (dekcats) על ידי יצירת מחלקה המורשת מ-nn.Module. הגדרת שכבה MTSL בודדת דורשת שלושה פרמטרים עיקריים: גודל הקלט (input_size), מספר יחידות זיכרון (hidden_size), ומספר שכבות (num_layers).
דוגמה בסיסית:

(2.1) lstm_layer = nn.LSTM(input_size = 1, hidden_size = 64, num_layers = 2)

כאשר הקלט מעוצב כטנסור בגודל [batch_size, seq_length, input_size], מערך MTSL עם batch_first=True מפיק פלט מצורה דומה, כאשר כל אלמנט בסדרה הופך לוקטור מימד hidden_size. תופעת ההדרגה הנעלמת (tneidarg gnihsinav), שהייתה בעיה ב-sNNR רגילות, מופחתת בצורה מובנית דרך שערים נטוש (setag tegrof) של MTSL, המאפשרים זרימת שיפוע עקבית על פני עשרות צעדי זמן.
בעבודה עם חילוץ אותות סינוסואידליים, הצינור הטיפוסי הוא:

1. Embedding Layer (אופציונלי) — לא רלוונטי לנתוני חידוש רציפים
2. Stacked LSTM — בדרך כלל 2–3 שכבות עם tuopord בין שכבות
3. Dense/Fully-connected — שכבת היציאה בגודל output_size
4. Activation — בדרך כלל זהות (ytiitnedi) או ReLU לצפוי רציף

היתרון של hcroTyP הוא שכל שכבה היא אובייקט פיתוני, ניתנת לחיבור והרחבה בקלות. מודל MTSLib מגדיל את מימד הפלט ל-hidden_size × 2 כי MTSL קדמי ו-MTSL אחורי פולטים וקטורים מובדלים המשורשרים (detanetacnoc) באופן עקבי.

2.2 הכנת נתונים ובניית sredaol

לפני הדרכה של כל מודל, סדרות זמן חדשות צריכות להיות מנורמלות ולעבד צורה סטנדרטית. נתונים גולמיים של אותות מעורבבים עם רעש דורשים:

1. **Normalization** — קנה מידה ל- $[-1, 1]$ או $[0, 1]$ בעזרת relacSxaMniM או erocs-Z
 2. **Windowing** — פיצול סדרות ארוכות לחלונות קטנים (בדרך כלל 215–821 דגימות)
 3. **Train-test split** — הפרדה זמנית (בעבור סדרות זמן, לא אקראית!) בדרך כלל 08–07
 4. **Batching** — הוצאת תת-קבוצות בגודל קבוע להדרכה מקבילה
- hcroTyP מספק את מחלקת Dataset וה-DataLoader לתהליך זה:

(2.2) `dataset = SignalDataset(noisy_signals, clean_signals)`

(2.3) `loader = DataLoader(dataset, batch_size = 32, shuffle = True)`

בנתוני סדרות זמן, `shuffle=True` עבור סט האימון מודגש אך חיוני לא לערבב את סט הבדיקה (שם אנו שומרים על סדר זמני). הבחירה של גודל חלון (`window_size`) משפיעה על איזון בין דגימות סטטיסטיות וביכולת של MTSL להיזכר בתלות זמן ארוכת טווח; חלונות קטנים מדי מביאים לאפילוג רועשים, וחלונות גדולים מדי עלולים לגרום לירידה בביצועים בשל משך תלות תיאורטי של MTSL.

2.3 הגדרת פונקציות `ssol` ו-`srezimitpoi`

בעיית חילוף אותות היא בעיית רגרסיה, כלומר אנו מנבאים ערכים עם ערכים ממשיים רציפים. שני ה-`ssol` נפוצים ביותר הם `Mean Squared Error (ESM)` ו-`Mean Absolute Error (EAM)`:

$$(2.4) \quad \text{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

$$(2.5) \quad \text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$$

כאשר $\hat{y}_i$ הוא הפלט החזוי של המודל ו- y_i היא התווית הנכונה (האות הנקי). `ESM` מעניש על שגיאות גדולות בצורה ריבועית, מה שהופכו לרגיש מאוד לחריגים, בעוד `EAM` יותר חסון ל-`sreiltuo` רצועות ליווי אל ערכי חריגים.

עבור חילוף אותות, בחרנו בדרך כלל `MSE` כי אנו רוצים לעקוב מדויק של הטופס מלא. אולם כדי לצמצם השפעה של דגימות רועשות עם שגיאות גדולות במיוחד, אנו יכולים להשתמש בהפסד חלק כמו `SmoothL1Loss (ssol rebuH)`:

$$(2.6) \quad \text{Huber}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| \leq \delta \\ \delta(|y - \hat{y}| - \frac{\delta}{2}) & \text{otherwise} \end{cases}$$

בחירת `optimizer` קריטית לחזוי הדרכה. `madA` (`torch.optim.Adam`) הוא הבחירה המעשית המובילה כיום:

(2.7) `optimizer = Adam(model.parameters(), lr = 0.001, betas = (0.9, 0.999))`

`madA` משלב מומנטום אדפטיבי עם שיעור למידה דינמי לכל פרמטר, מה שהופכו לשיטה מקבילה לאופטימיזציה ברוב ההגדרות. האלטרנטיבה, `DGS` עם מומנטום, דורשת כיוול ידני יותר אך יכולה לתת צפויות רעות הטובות יותר במקרים מסוימים.

שיעור הלמידה (`learning_rate`) בדרך כלל מתחיל בערך בטווח 10^{-4} עד 10^{-2} , עם `learning rate scheduling` מומלץ (למשל `StepLR` או `ReduceLRonPlateau`) להוריד את קצב הלמידה כשה-`ssol noitadilav` מפסיק להשתפר.

2.4 לולאות הדרכה והערכה

לולאת הדרכה סטנדרטית (pool gniniart) בפיתוח hcroTyP עוקבת אחר תבנית:

1. **ssap drawroF** — העבר אצימות דרך המודל כדי לחזות פלטים
2. **ssol etupmoC** — חשב את ה-ssol בין חזויים לנכון
3. **ssap drawkcaB** — חשב שיפועים בעזרת dargotua
4. **pets rezimitpO** — עדכן משקלים בהתאם לשיפועים
5. **stneidarg raelC** — אפס את הגרדיאנטים להתחלה נקייה בצעד הבא

בצורה יותר רשמית:

```
for epoch in range(num_epochs):
    for batch_idx, (noisy, clean) in enumerate(train_loader):
        # Forward
        predictions = model(noisy)
        loss = criterion(predictions, clean)

        # Backward
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

בדרך כלל, אנו גם מחשבים validation loss בכל זמן או בסוף כל hcope כדי לבדוק overfitting. ה-gnittifrevo מתרחש כאשר המודל למדה לזכור את נתוני ההדרכה במדויק אך נכשל להכליל לנתונים חדשים (tset). ניתן להימנע מזה באמצעות:

1. **gnippots ylraE** — עצור את ההדרכה כשה-ssol noitadilav מפסיק להשתפר
2. **tuopord** — הוסף sreyal tuopord (בדרך כלל 5.0–2.0 etar pord) בין שכבות MTSL
3. **noitaziralugeR** — הוסף 2L noitaziralugeR (yaced thgiew) ל-rezimitpo
4. **gnitniopkcehc ledom** — שמור את המשקלים הטובים ביותר לפי cirtem noitadilav

עבור מטרות חילוץ אותות, מדדי הערכה טיפוסיים הם:

$$(2.8) \quad \text{SNR} = 10 \log_{10} \frac{\sum_i (y_i^{\text{true}})^2}{\sum_i (y_i^{\text{true}} - y_i^{\text{pred}})^2}$$

$$(2.9) \quad \text{NMSE} = \frac{\sum_i (y_i^{\text{true}} - y_i^{\text{pred}})^2}{\sum_i (y_i^{\text{true}})^2}$$

$$(2.10) \quad \text{Correlation} = \frac{\text{Cov}(y^{\text{true}}, y^{\text{pred}})}{\sigma_{y^{\text{true}}} \sigma_{y^{\text{pred}}}}$$

RNS משפר כאשר השגיאה בניבוי קטנה בהשוואה להספק האות. ESMN היא שגיאה מנורמלת בטווח $[0, \infty)$ כאשר 0 הוא ניבוי מושלם. מקדם המתאם מודד קיום ליניארי בין החזוי לנכון, עם ערכים כמו 59.0 מסמלים קבלה טובה של צורה (אף כי השיעור או הסטה יכולים להיות שגויים). בסיום הדרכה, מקובל לדווח על ממוצע של מדדים אלה על פני סט הבדיקה השלם, עם פירוק אופציונלי לפי טווח RNS קלט (למשל "ביצוע בעל RNS = Bd 0" בהשוואה ל-"RNS = Bd 02"). זה עוזר להבין איך המודל משתנה בתנאים קשים יותר.

פרק 3

פרוטוקול בקרת המודל

3.1 אינטרפייס של Model Context Protocol

פרוטוקול בקרת המודל (MCP, Model Context Protocol) הוא ממשק סטנדרטי המעניק יכולת ליישומים חיצוניים לתקשר עם מודלים גדולים של שפה, ובכך להוציא צו מעבר שפה, כדי לשלוט בהקשר שלהם. MCP מגדיר דרך מסודרת לקריאה למודל עם כלים או משאבים שאינם חלק מניהול שחקן של המודל עצמו.

בחברה של מודלים כגון Claude, OpenAI API, ו-Google PaLM, יש צורך בתקן אונייני שמאפשר יישומים שונים להיות משודרים לתקשוריות דומות. MCP מהווה סט של כללים וכלים שמספקים יכולת זו.

טבלה 3.1: השוואת מודלי שפה מרכזיים

Model	Parameters	Score	Notes
BERT-base	110M	88.5 GLUE	Encoder-only
GPT-3	175B	64.3 BIG-G	Decoder-only
T5-large	770M	89.7 GLUE	Encoder-Decoder
LLaMA-2-7B	7B	63.2 MMLU	Decoder-only

3.2 טכניקות שדרוג קשר

קשר בתוך מודלים יכול להתהווה בדרכים שונות. אחת מן הטכניקות הנפוצות ביותר היא prompt engineering — כתיבה של הנחיות ויעדים בדרך שמזמינה את המודל לייצור פלטים חדשים ובעלי ערך. אחרת, אנחנו יכולים להשתמש בשיטות כגון in-context learning, שם אנו משדרים דוגמאות של קלט וחציצי פלט כדי ללמד את המודל על הדפוס שאנחנו רוצים.

מודל גדול כגון GPT-3 יכול לקבל מאות אלפים של טוקנים (תווים או מילים) בקלט שלו בעת אחת. זה מפתח הזדמנויות חדשות לשדרוג קשר, אך גם מציב אתגרים. הנהלה של קשר כזה דורשת חישובים משמעותיים, וגם יצירת אסטרטגיה לאילו מידע להכניס למודל ובאיזה סדר.

3.3 ביטחון וערכיות בשדרוג קשר

יש חשיבות גבוהה להבטיח שהקשר שנשדרג למודל הוא בטוח, במבחן שהמודל לא יחשוף מידע חסוי או לא יביא לתוצאות שאינן רצות. זה דורש בדיקות קפדניות של הנתונים המוקדשים, ובדיקות של אם

המודל מנהל את הנתונים בדרך בטוחה.
יותר מכך, Anthropic (היצרנית של Claude) ובעלות אחרות משפרות כל הזמן את השיטות שלהם לשדרוג קשר בטוח. טכניקות כגון constitutional AI וקריאה לפנים של מודלים עוזרות להבטיח שהקשר מנוהל בעיקרון של בטיחות ודיוק.

פרק 4

איומים וחולשות

4.1 סוגי איומים על מודלים של שפה

כיום, כאשר מודלים של שפה משמשים יותר ויותר בתחומים רגישים כגון בריאות, כספים, וחינוך, סוגיות של בטיחות והיכחדות קריטיים. יש מספר סוגים של איומים שעלול להיתקל בהם בעת השימוש במודלים כאלה:

1. Hallucinations: המודל יוצר מידע שאינו נכון או שלא קיים בנתונים ההדרכה.
2. Prompt injection: משתמש זדוני יכול לתחברק את הקלט כדי להכניס הוראות מסתירות הגורמות למודל להתנהג בצורה לא רצויה.
3. Data poisoning: כאשר מישהו מיתח את נתוני ההדרכה כדי להשפיע על התנהגות המודל.
4. Adversarial examples: קלט מעוטר בדרך ערמומית כדי לגרום למודל לטעות בדרך מסוימת.
5. Privacy leakage: המודל יכול לגלות מידע רגיש שהיה בנתוני ההדרכה.

4.2 איומים על פרוטוקול בקרת המודל

כאשר משתמשים ב-MCP (פרוטוקול בקרת המודל), נוצרים איומים נוספים שקשורים לתקשורת בין הישום ובין המודל. למשל, אם המודל יקבל הוראה מיישום חיצוני שלא עבר בדיקה, הוא יכול להתנהג בדרך לא בטוחה.

בנוסף, ה-context window של מודל (גודל הקלט המרבי שהוא יכול לקבל) יכול להיות נוצל בצורה שלילית. מודל כגון Claude יכול לקבל עד כ-002 אלף טוקנים בקלט, מה שפותח אפשרויות להשפעות בקשר ישירות.

סוגיה משמעותית נוספת היא זו של authorization: כיצד נוודא שגם כשמודל משתמש בכלים או מידע חיצוני דרך MCP, הוא עדיין משתמש בהן בדרך המתאימה לרמת הזכויות של משתמש מסוים?

4.3 אתגרים של קנה מידה

ככל שמודלים הופכים יותר גדולים וכוחניים, אתגרים של בקרה הופכים יותר גדולים. מודל כגון GPT-3 עם 571 מיליארד פרמטרים יכול להפוך כל הנוף של סוגיות. האתגרים הוא לא רק לבנות בדיקות טובות, אלא גם לבנות אלה שנשמרים עם הגדלת המודל עצמו.

כמו כן, יש צורך לפתח גישות למידה שלא דורשות מליליוני שעות של בדיקה אנושית, אלא מאמצות שמודלים עצמם יכולים להשתמש בהם לבדיקה ולשיפור שלהם.

פרק 5

הפחתת סיכונים וכיווניות עתידית

5.1 אסטרטגיות הפחתת סיכונים

על מנת להפחית סיכונים בעת שימוש במודלים של שפה, חברות ועמותות חייבות לתכנן מסגרת שוטפת של בדיקה והשגחה. זה כוללת מספר אסטרטגיות:

1. Red-teaming: קבוצה של בודקים מנסים לחזות את הדרכים שבהן יכול מודל להיכשל, וכך לזהות חולשות.
2. Continuous monitoring: עקוב קבוע על התנהגות המודל בסביבה של ייצור כדי לזהות תופעות לא רצויות.
3. Fallback mechanisms: ניסיון לספק דרכים אלטרנטיביות או בדיקות מגניבה כאשר מודל לא בטוח.
4. Fine-tuning and adaptation: התאמה של המודל לסביבה ספציפית, באמצעות ייתרון או יישום של LoRA (noitapdaA knaR-woL).
5. User feedback loops: איסוף משוב מהמשתמשים הסופיים כדי לשפר את המודל וללמוד על סוגיות חדשות.

5.2 תקנון וחקיקה

ככל שמודלים של שפה גדולים הופכים לחלק משכיח יותר של הציבור, ממשלות וגופים בינלאומיים מתחילים להתעניין בתקנון של תחום זה. כבר קיימות דוגמאות כגון EU AI Act, אשר מעמיד כללים על מודלים של בינה מלאכותית בהקשר אירופאי. תקנון יכול לכלול דרישות על:

- תיעוד של נתוני ההדרכה ושיטות
- בדיקות של הטיות וביצועים
- הודעות שפופה למשתמשים על שימוש במודלים
- זכות למחוק מידע אישי מנתוני הדרכה

חברות כמו OpenAI, Anthropic, וגוגל משתתפות בדיונים אלה כדי להגדיר תקנים ושיטות בטיחות בתחום.

5.3 כיווניות עתידית ומחקר פתוח

התחום של בינה מלאכותית גדולה והשגחה עליה עדיין בשלבי ניצנים. מחקר עתידי יכול להתמקד בכמה תחומים:

1. Interpretability: הבנה של איך מודלים מקבלים החלטות ביותר עמוקה.
 2. Efficiency: פתוח מודלים יותר קטנים וחסכוניים בחישוב שעדיין חזקים.
 3. Multilingual models: משפרים קודים של מודלים שיכולים להתמודד עם שפות שונות, כולל שפות RTL כמו עברית.
 4. Cross-modal learning: שילוב של טקסט, תמונות, וסוגי מדיה אחרים.
 5. Human-in-the-loop systems: מערכות שנותנות לאדם בקרה וביקורת על תוך כדי.
- מודלים כגון LLaMA, Mixtral, ו-Phi מציגים טרנדים של דחיקה של גבולות היעילות. כלים כגון MCP (פרוטוקול בקרת המודל) וטכניקות בקרת קשר מתקדמות יאפשרו יישומים ועסקים לנצל יכולות אלה בדרך בטוחה, חסכונית, ותמידית.
- אנו בתחילת סוף מהפכה בתחום עיבוד השפה הטבעית, וה-Transformer מודלים הם בליבת מהפכה זו. התמודדות עם האתגרים של בטיחות, בקרה, ודיוק יהיה מרכזי לעתידות בוטחות ומשפחתיות של תחום זה.

נספח א': מילון מונחים

נספח זה מרכז את המונחים המרכזיים המופיעים בעבודה, תוך מתן הגדרות תמציתיות לשימוש עתידי.

Attention Mechanism (מנגנון תשומת לב) תהליך חישובי המאפשר לכל אסימון (token) בנתון קלט לקבוע את מידת הרלוונטיות של יתר האסימונים בייצוג התוצאה. מחושב באמצעות מטריצות שאילתה (Query), מפתח (Key) וערך (Value).

Transformer ארכיטקטורת רשת נוירונית המבוססת אך ורק על מנגנון תשומת לב, ללא רשתות רקורנטיות. הוצגה לראשונה במאמר "Attention Is All You Need" (7102) והפכה לבסיס של רוב מודלי השפה המודרניים.

(LLM) Large Language Model מודל שפה גדול המאומן על כמויות עצומות של טקסט, המסוגל לבצע מגוון רחב של משימות שפה ללא כוונן ייעודי. דוגמאות: Gemini, Claude, GPT-4.

(MCP) Model Context Protocol תקן פתוח המגדיר ממשק אחיד בין מודלי שפה לכלים חיצוניים. מאפשר לסוכן לגלות, לפנות ולהשתמש בכלים (שרתים) בצורה עקבית ובטוחה.

ReAct (חשיבה-פעולה) מסגרת לסוכני LLM המשלבת שלבי חשיבה מפורשים (Reasoning) עם שלבי פעולה (Acting) בלולאה חוזרת. מאפשרת לסוכן לנמק לפני שהוא מבצע קריאה לכלי.

Reflexion שיטה לשיפור עצמי של סוכנים על בסיס משוב מילולי. הסוכן מסכם את כישלונותיו הקודמים ומאחסן אותם בזיכרון, ומשתמש בהם לשיפור ניסיונות עתידיים.

(CoT) Chain-of-Thought טכניקת הנחיה המעודדת את המודל לפרט את שלבי החשיבה הביניים לפני מתן התשובה הסופית, ובכך משפרת ביצועים במשימות מורכבות.

Toolformer מודל שפה שאומן לשלב באופן אוטונומי קריאות API בתוך יצור הטקסט שלו. המודל לומד מתי כדאי לקרוא לכלי ואיך לפרש את תוצאותיו.

AvaTaR מסגרת רב-סוכנית המשתמשת בלמידה מחיזוק להכשרת סוכני כלים. בכל איטרציה, סוכן מתאמן (optimizer) משפר את מדיניות השימוש בכלים של הסוכן הראשי.

DEPS (תכנון מודע-תלויות) שיטת תכנון המגדירה גרף תלויות בין תת-משימות ומבטיחה ביצוע בסדר הנכון, תוך ניהול תלויות נתונים בין שלבים.

BiDi (דו-כיוונית) טכנולוגיה לעיבוד טקסט המשלב כיוונים שונים: ימין-לשמאל (עברית) ושמאל-לימין (אנגלית, נוסחאות). מוטמעת בהפקת המסמך באמצעות חבילות polyglossia ו-luabidi.

Fine-Tuning (כוונן עדין) תהליך אימון נוסף של מודל קיים על מערך נתונים ממוקד, במטרה להתאים את התנהגותו למשימה ספציפית מבלי לאמן מאפס.

Hallucination (הזיה) תופעה שבה מודל שפה מייצר מידע שגוי שנראה סביר, אך אינו מבוסס על נתוני האימון או על המציאות. אחד האתגרים המרכזיים בפריסת LLM בסביבות ייצוריות.

ביבליוגרפיה